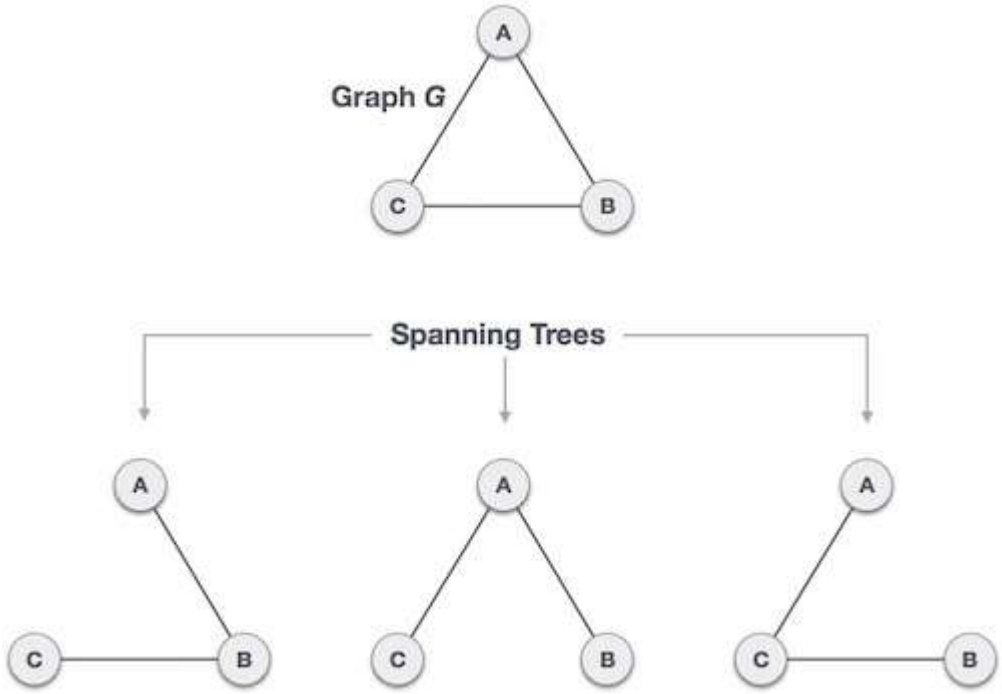
Minimum Spanning Tree

Tree and Spanning Tree

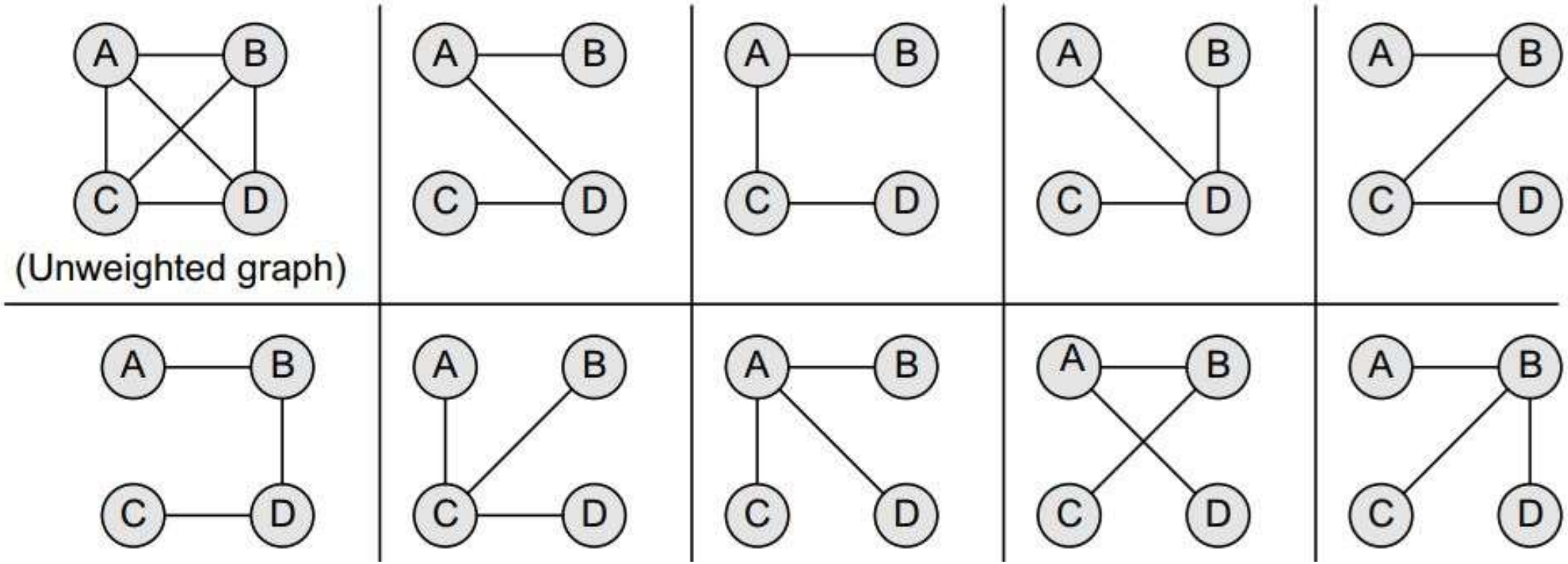
- Tree: A connected acyclic graph
- Given a connected and undirected graph, a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together.



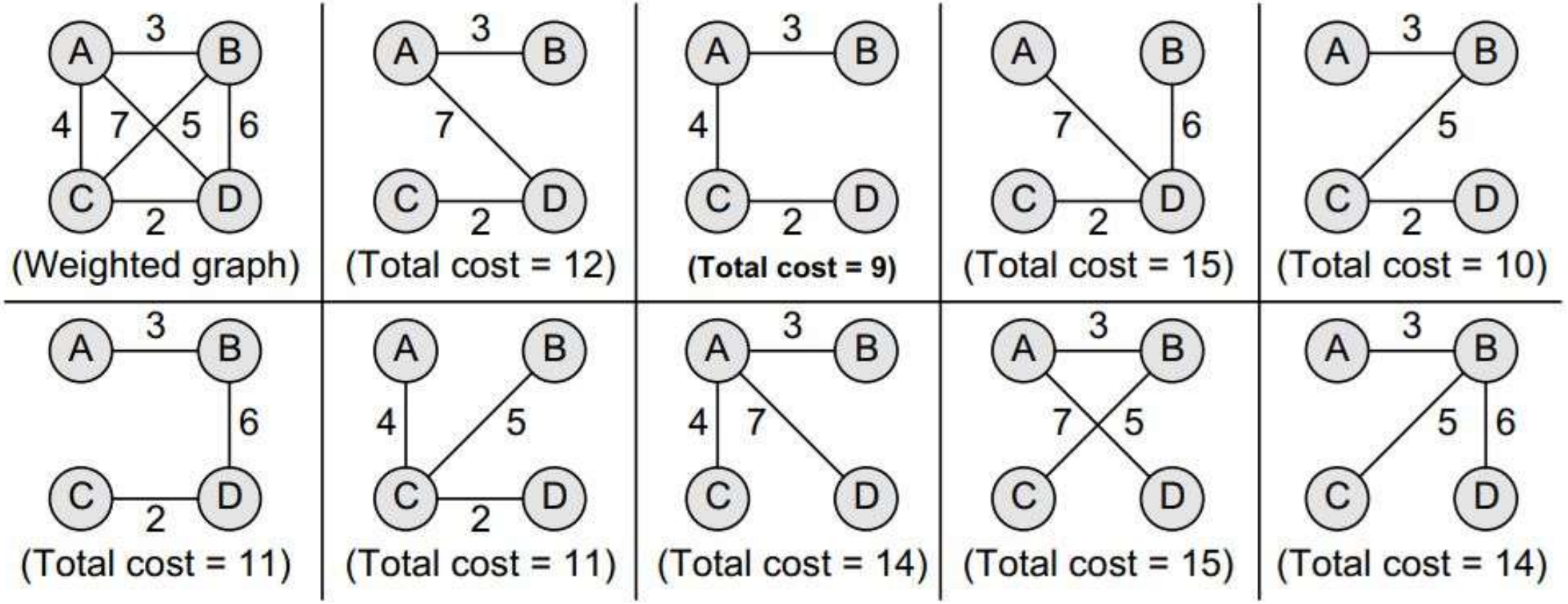
Minimum Spanning Tree (MST)

- A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected, undirected graph is a spanning tree with a weight less than or equal to the weight of every other spanning tree.

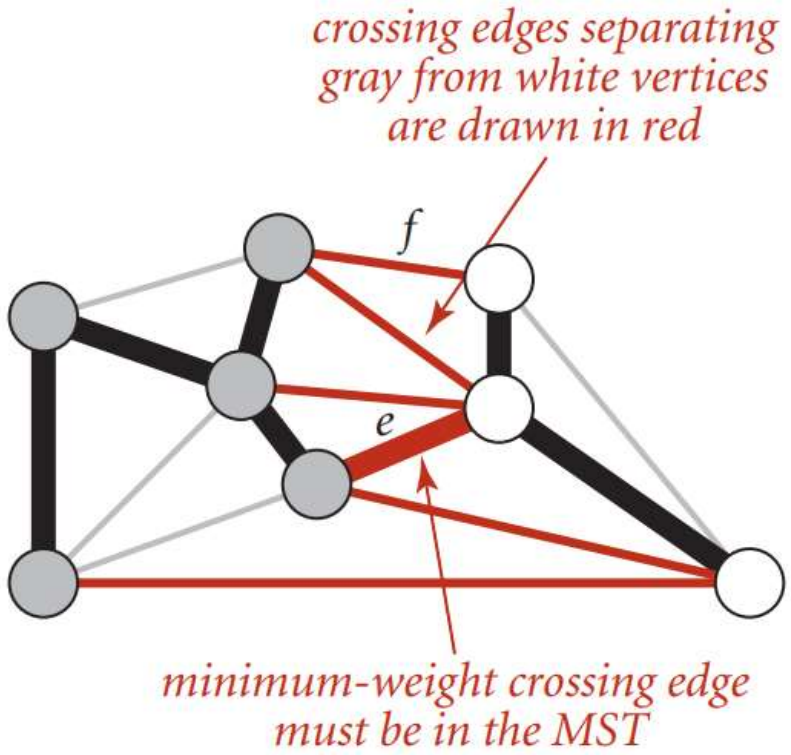
MST Example (Unweighted Graph)



MST Example (Weighted Graph)



Prims Algorithm (Idea)



Cut property

Terminology

- ***Tree vertices*** Vertices that are a part of the minimum spanning tree T.
- ***Fringe vertices*** Vertices that are currently not a part of T, but are adjacent to some tree vertex.
- ***Unseen vertices*** Vertices that are neither tree vertices nor fringe vertices fall under this category.

Prims Algorithm

```
Step 1: Select a starting vertex
Step 2: Repeat Steps 3 and 4 until there are fringe vertices
Step 3:     Select an edge e connecting the tree vertex and
            fringe vertex that has minimum weight
Step 4:     Add the selected edge and the vertex to the
            minimum spanning tree T
        [END OF LOOP]
Step 5: EXIT
```

Figure 13.30 Prim's algorithm

Prims Algorithm

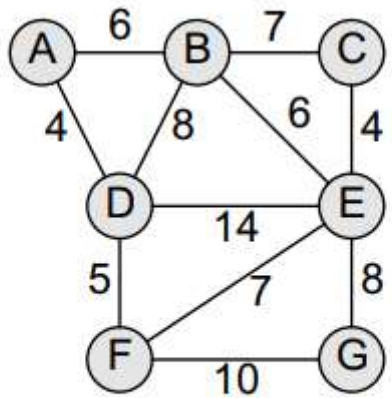


Figure 13.32 Graph G

Prims Algorithm

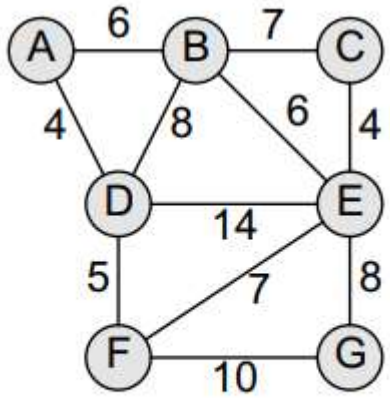
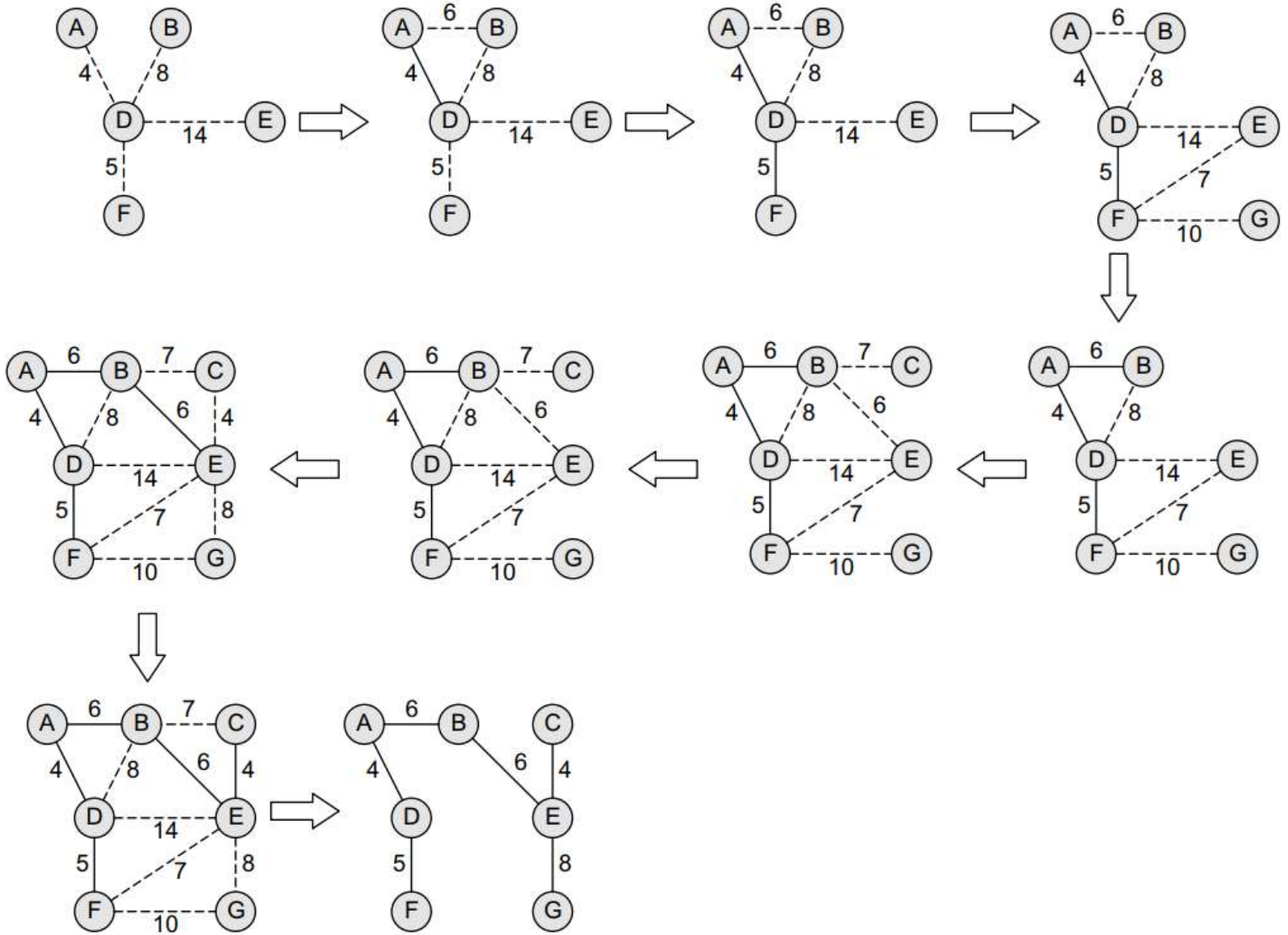


Figure 13.32 Graph G



Prims Algorithm (Implementation)

- Create a set *(mstSet)* that keeps track of vertices already included in MST.
- Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign the key value as 0 for the first vertex so that it is picked first.
- While mstSet doesn’t include all vertices
  - Pick a vertex *u* which is not there in *mstSet* and has a minimum key value.
  - Include *u in the* mstSet.
  - Update the key value of all adjacent vertices of *u*.

Pseudo Code

```
PRIM(G, w, r):
 for each u in G:
   u.key = INF
   u.p = NIL
 r.key = 0
 Q = G
 while Q is not empty:
   u = EXTRACT-MIN(Q)
   for each v in Adj[u]:
     if v in Q and w(u, v) < v.key:
       v.p = u
       v.key = w(u, v)
 return G
```

Time Complexity

Pseudo Code

```
PRIM(G, w, r):
 for each u in G:
   u.key = INF
   u.p = NIL
 r.key = 0
 Q = G
 while Q is not empty:
   u = EXTRACT-MIN(Q)
   for each v in Adj[u]:
     if v in Q and w(u, v) < v.key:
       v.p = u
       v.key = w(u, v)
 return G
```

1
2
3
4
5

- 1: O(V)
- 2: O(1)
- 3: Runs V times
- 4: O(log V)
- 5: O(log V), runs 2E times altogether
  - (k1*logV + k2*logV+…+kn*logV)
  - = logV(k1+k2+…+kn)
  - =logV*2E
  - O(E*logV)

Total runtime: O(V) + O(1) + O(V*logV) + O(E*logV)

Difference with Dijkstra

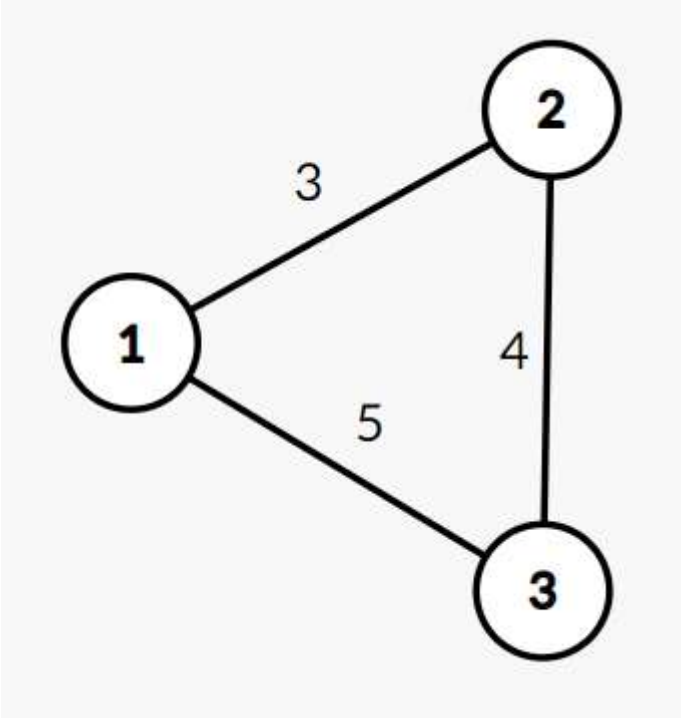
In Dijkstra, we choose an edge(u,v) as part of the shortest path:

If cost[u] + weight(u,v) < cost[v]

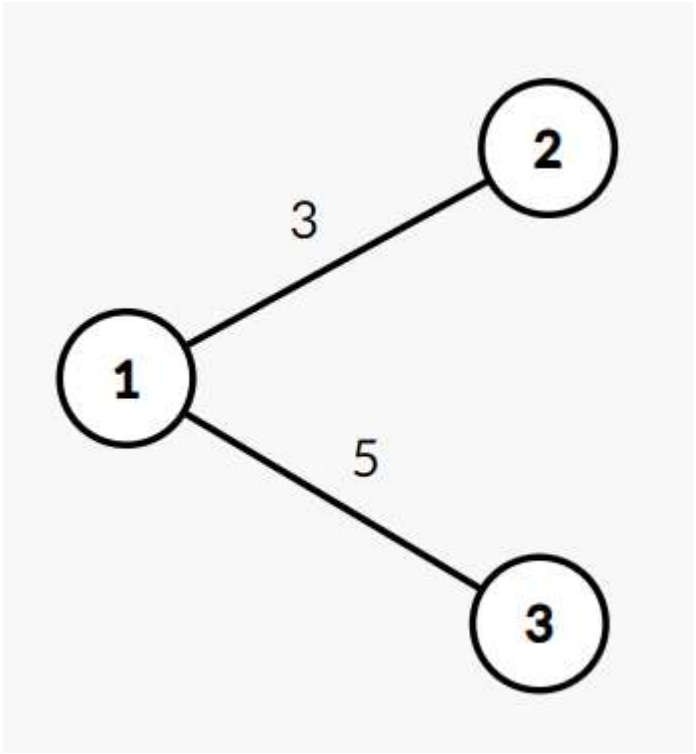
In Prim's algorithm, we choose an edge(u,v) as part of the MST:

If weight(u,v) < cost[v]

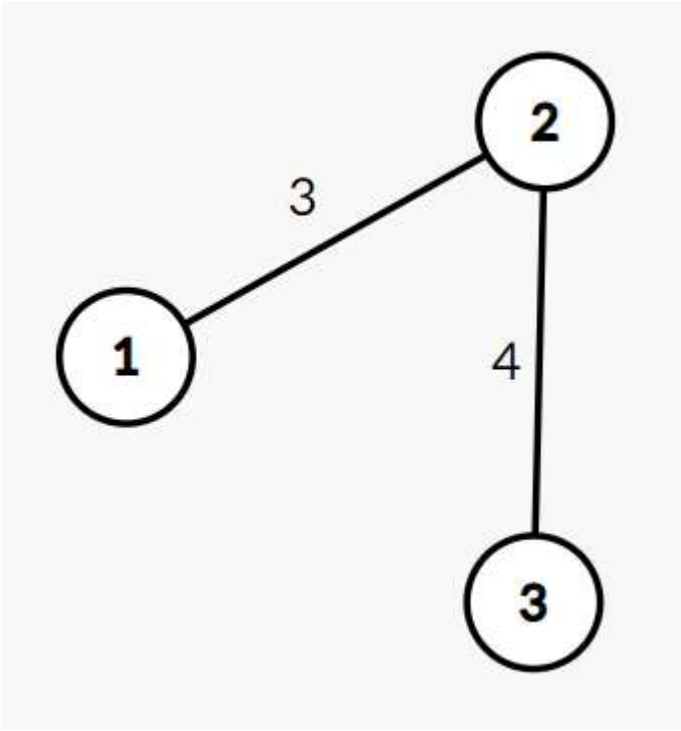
Example of difference



Graph



Shortest Path (from 1)

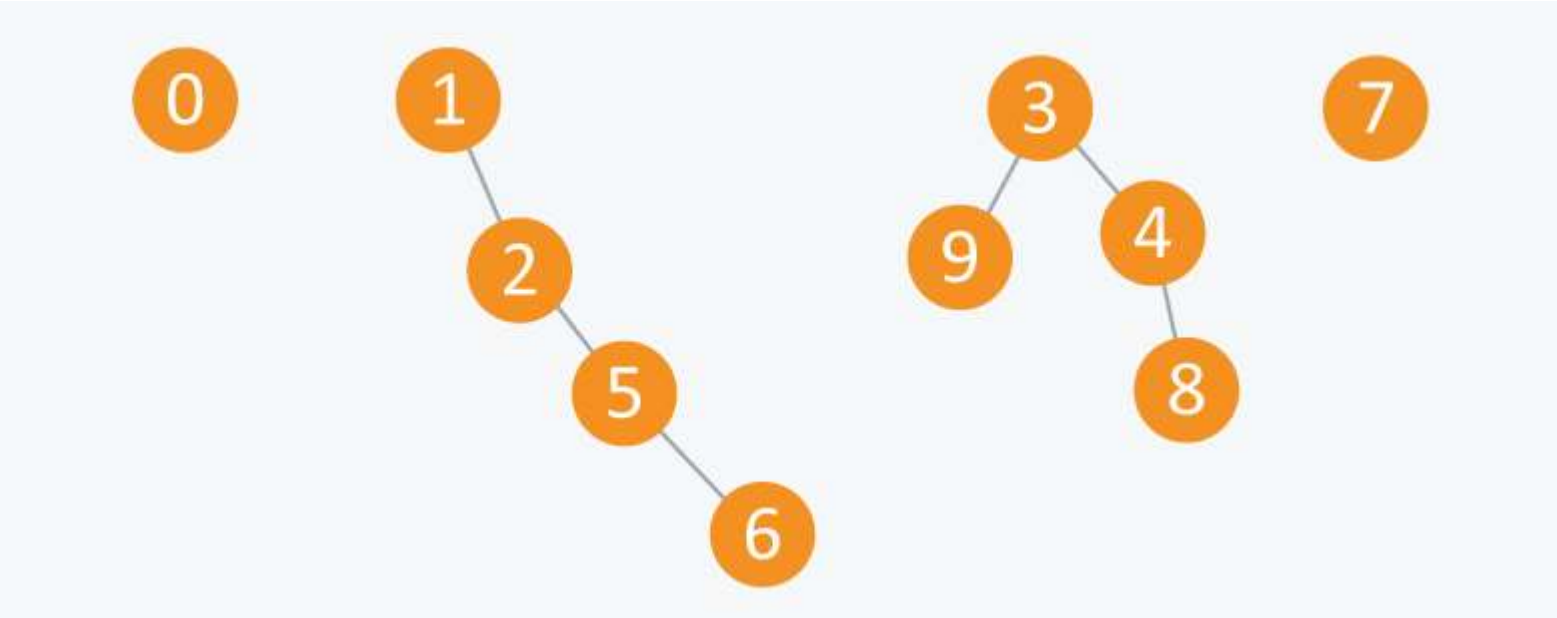


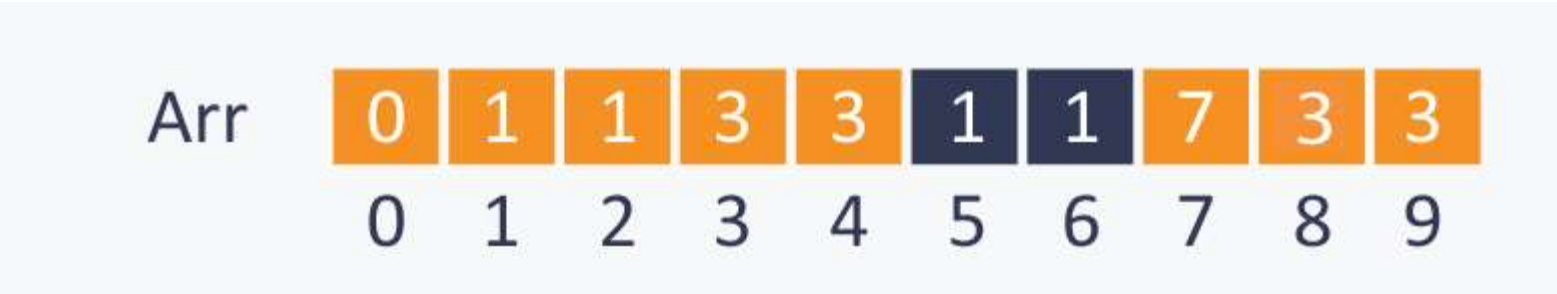
MST (calculated from 1)

Kruskal’s MST Algorithm

1. Sort all the edges in increasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are (V-1) edges in the spanning tree.

Union-Find





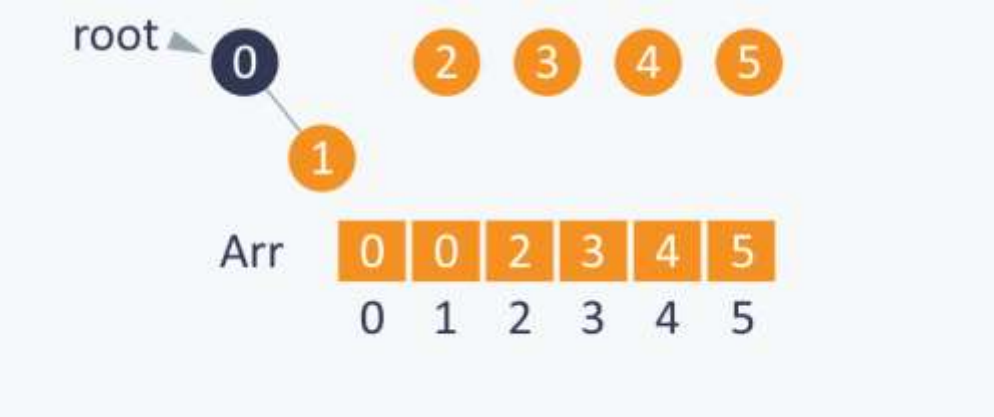
Implementation

```
void union(int Arr[ ], int N, int A, int B)
{
    int TEMP = Arr[ A ];
    for(int i = 0; i < N;i++)
    {
        if(Arr[ i ] == TEMP)
        Arr[ i ] = Arr[ B ];
    }
}
```

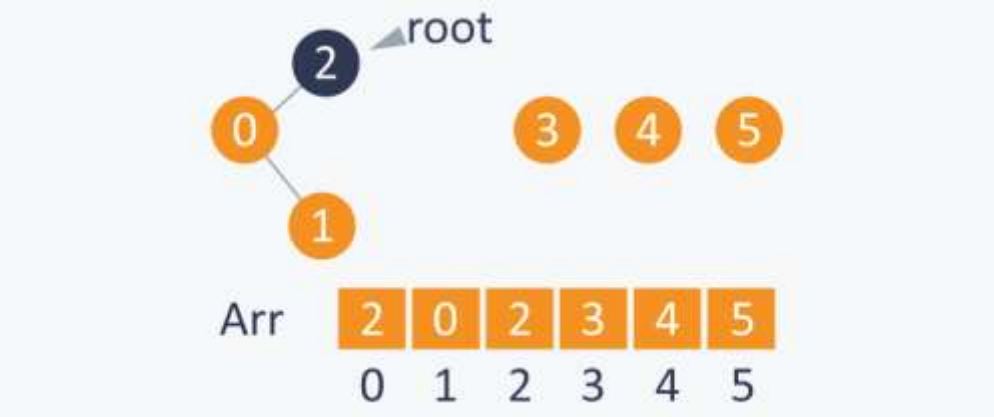
Implementation

```
 bool find( int Arr[ ], int A, int B)
{
        if(Arr[ A ] == Arr[ B ])
                return true;
        else
                return false;
}
```

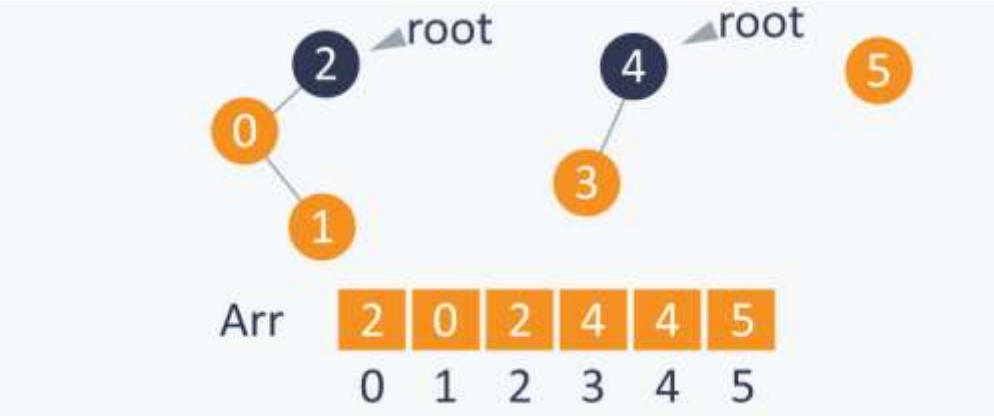
Union-Find



Union (1,0)

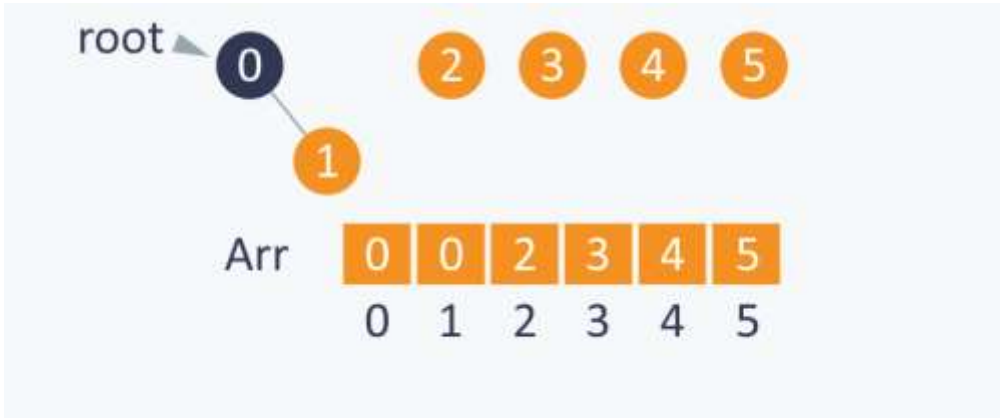


Union (0,2)

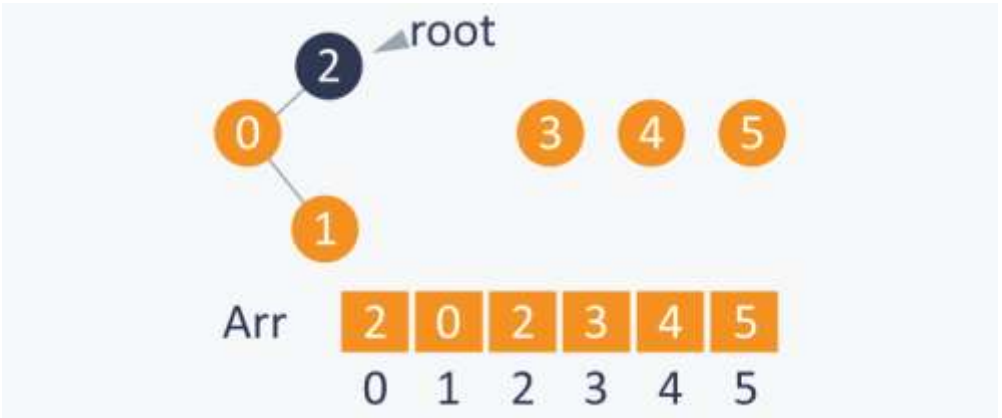


Union (3,4)

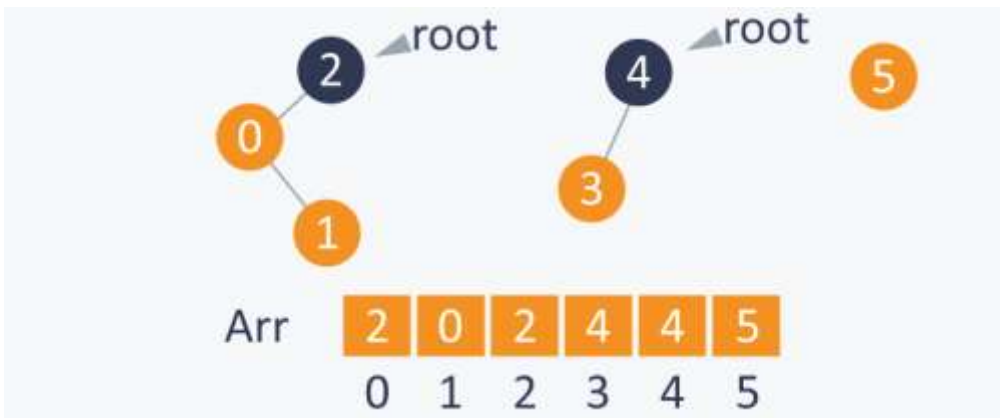
Union-Find



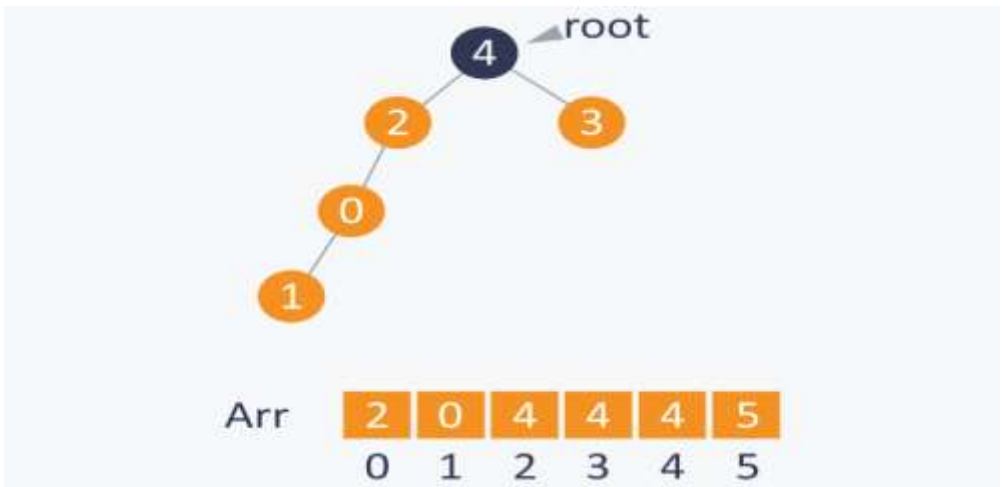
Union (1,0)



Union (0,2)



Union (3,4)



Implementation

```
int root(int Arr[ ],int i)
{
    while(Arr[ i ] != i) {
            i = Arr[ i ];
    }
    return i;
}
```

Implementation

```
int union(int Arr[ ] ,int A ,int B)
{
    int root_A = root(Arr, A);
    int root_B = root(Arr, B);
    Arr[ root_A ] = root_B ;
}
```

```
bool find(int A,int B)
{
    if( root(A)==root(B) )
            return true;
    else
            return false;
}
```

Ranked Union

During union, set the smaller sized set’s root as the large set’s root

Path Compression with ranked union

During find operation, set the root for all the seen nodes as the parent root

Runtime Complexity

- Sorting: E*LogE
- Union-Find O(V) or O(LogV) or O(1) [Basic, Ranked, Path-Compression)

Total runtime: O(E*LogE) + O(E*V) or O(E*LogV) or O(E)

References:

Union-Find:

- https://www.hackerearth.com/practice/notes/disjoint-set-union-union-find/